

Documentation

Summary

This project provides a high-performance pipeline for estimating Transfer Entropy (TE) between continuous-time event sequences (e.g., neural spike trains) using deep learning. It uses Recurrent Mixture Density Networks (RMDNs) to model the conditional density of inter-event intervals (inter-spike intervals), enabling robust estimation of the TE rate (nats per second) for processes with long temporal dependencies.

Additionally, the project offers scripts for HPC parallelization to facilitate high-throughput analysis, including the estimation of transfer entropy between two groups of neurons (e.g., BC and POM neuron populations).

Theoretical Foundation

Transfer entropy quantifies the amount of predictive information the history of the source process (X) could provide about the current state of the target process (Y).

In the thesis, Tuoxing derived the formula for transfer entropy rate expressed as the difference between two log-density ratios of target inter-event intervals (IEIs):

$$TE_{X \rightarrow Y}(k, l) := \lim_{t \rightarrow \infty} \frac{1}{t - t_0} \sum_{i=1}^{N_Y([t_0, t])} \left[-\ln \frac{p_Y(\tau_Y(i) \mid \tau_Y^{(k)}(\leq i))}{\int_{\tau_Y(i+1)}^{\infty} p_Y(u \mid \tau_Y^{(k)}(\leq i)) du} + \ln \frac{p_Y(\tau_Y(i+1) \mid \tau_Y^{(k)}(\leq i), \tau_X^{(l)}(\leq N_X(y_i)), y_i - x_{N_X(y_i)})}{\int_{\tau_Y(i+1)}^{\infty} p_Y(u \mid \tau_Y^{(k)}(\leq i), \tau_X^{(l)}(\leq N_X(y_i)), y_i - x_{N_X(y_i)}) du} \right]$$

The TE estimation problem is then converted to two prediction problems:

1. **(Reduced Model)** Use only the historical inter-event intervals of the **target** to predict the PDF of the next inter-event interval.
2. **(Full Model)** Use the historical IEIs of both the **source and the target** to predict the PDF of the next inter-event interval.

Two RMDNs are trained **separately** for these two tasks. After training, each model is evaluated on the ground-truth intervals to obtain per-event log-probabilities. The transfer entropy rate is then computed by averaging the log ratio over all events in the target process according to the formula above:

$$TE = \text{mean}(\ln_{yyx}) - \text{mean}(\ln_{yy})$$

The parameters **k** and **l** in the formula specify the number of historical IEIs considered. Theoretically, larger **k** and **l** yield more accurate estimates, but they also increase the dimensionality of the conditioning variables and thus the amount of data required.

Project Architecture

```
.
├── data/                # Input datasets (HDF5 and MAT files)
├── demo/                # Demonstration notebook and results
├── documentation/       # This documentation
├── dpp/                 # Deep point process model utilities
├── hpc/                 # HPC parallelization scripts
├── results/             # Output directory for computed results and plots
├── src/                 # Primary source code for TE estimation
├── environment.yml       # Conda environment specification
├── requirements.txt      # Full pip dependency list
└── setup.py             # Package installation configuration
```

data/

Directory for input datasets.

- `event_times_data.h5`: HDF5 file containing event times organized by neuron group and neuron ID (generated from `testFile.mat` via `hpc/mat_to_h5.py`).
- `testFile.mat`: Raw MATLAB spike data file (source for the HDF5 file above).

demo/

Contains the demonstration notebook (`demo.ipynb`) outlining the full estimation pipeline for a single local run. A pre-rendered `demo.html` and example result plots are also included.

dpp/

Dependent module for deep point process model components, including data batching (`batch.py`), dataset handling (`dataset.py`), sequence utilities (`sequence.py`), and distribution definitions (`distributions/`).

src/

Primary source code for the transfer entropy estimation framework.

- `src/te_tpp.py`: Core source file containing the end-to-end single-estimation pipeline. Includes data preprocessing, training of `LogNormMix` models, and the `Estimate_TE_Hazard` function which implements the estimation algorithm.
- `src/CoTETE.py` / `src/CoTETE_example_*.py`: Scripts to generate synthetic data and compare this method with CoTETE, the leading non-parametric TE estimation method ([CoTETE.jl](#)).

hpc/

Source code for scaling the estimation framework on HPC clusters. See the [HPC Workflow Pipeline](#) section for a step-by-step usage guide.

File	Description
<code>hpc_header.py</code>	Shared utility functions and task list generation for SLURM array jobs
<code>mat_to_h5.py</code>	Converts raw MATLAB <code>.mat</code> files to HDF5 format
<code>opt_reduced_model_batch.sh</code>	SLURM script for reduced model hyperparameter optimization
<code>opt_full_model_batch.sh</code>	SLURM script for full model hyperparameter optimization
<code>hyper_opt_reduced.py</code>	Optuna-based hyperparameter search for the reduced model
<code>hyper_opt_full.py</code>	Optuna-based hyperparameter search for the full model
<code>run_reduced_model_batch.sh</code>	SLURM script for multi-run estimation with the reduced model
<code>run_full_model_batch.sh</code>	SLURM script for multi-run estimation with the full model
<code>multi_runs_reduced.py</code>	Runs repeated reduced model estimations for a single task
<code>multi_runs_full.py</code>	Runs repeated full model estimations for a single task
<code>aggregate_results.py</code>	Consolidates all per-task CSV results into a single HDF5 file
<code>calculate_te.py</code>	Computes final TE rates with statistics and confidence intervals
<code>plot_te_heatmap.py</code>	Generates heatmap visualizations of pairwise TE estimates
<code>check_results.py</code>	Verifies completeness of optimization and multi-run outputs
<code>check_env.py</code>	Checks the environment and GPU availability
<code>debug.py</code>	Debugging utilities

results/

Output directory for computed results and plots. Structured as follows after a full HPC run:

```

results/
├─ opt-reduced/           # Per-task best hyperparameter configs (reduced
model)
├─ opt-full/             # Per-task best hyperparameter configs (full model)
├─ runs-reduced/         # Per-task multi-run CSV results (reduced model)
├─ runs-full/           # Per-task multi-run CSV results (full model)
├─ multi_runs_results.h5 # Aggregated results (output of
aggregate_results.py)
└─ te_results_hpc.csv    # Final TE estimates with statistics (output of
calculate_te.py)

```

Main Functions in `src/te_tpp.py`

`TE_estimation_tpp`

The core function implementing a single transfer entropy rate estimation.

```
TE_estimation_tpp(event_time, configs: dict, seed: int = 42, trial=None)
```

Inputs

- `event_time`: A list of `torch.Tensor` arrays containing absolute event times in seconds. The **first** tensor is the **target** process and the **second** is the **source** process. Example:

```
# Source fires at 1, 3, 5 s; target fires at 2, 4, 6 s
event_time = [torch.tensor([2., 4., 6.]), torch.tensor([1., 3., 5.])]
```

- `configs`: A dictionary containing all model and training hyperparameters:

```
configs = {
    "model_config_yyx": {
        "model_name": "yyx",          # Arbitrary name string
        "context_size": 16,           # RNN hidden vector size
        "num_mix_components": 1,       # Number of mixture components
        "hidden_sizes": [4, 32],      # MLP hidden layer sizes
        "context_extractor": "gru",    # RNN type: "gru", "lstm", or "mlp"
        "activation_func": "GELU",     # Activation: "GELU", "ReLU", "Tanh"
    },
    "train_config_yyx": {
        "L2_weight": 1e-4,            # L2 regularization weight
        "L_entropy_weight": 0,         # Entropy regularization weight
        "L_sep_weight": 0,            # Separation regularization weight
        "L_scale_weight": 1e-4,        # Scale regularization weight
        "learning_rate": 5e-4,         # Adam optimizer learning rate
        "max_epochs": 1000,            # Maximum training epochs
        "display_step": 5,             # Log interval (epochs)
        "patience": 40,               # Early stopping patience (epochs)
    },
    "model_config_yy": {
        "model_name": "yy",
        # ... same keys as model_config_yyx ...
    },
    "train_config_yy": {
        # ... same keys as train_config_yyx ...
    },
    "data_prep_config": {
        "batch_size": 128,             # Sequences per training batch
        "shuffle": False,              # Whether to shuffle source sequence
        "total_time": 300,             # Total duration in seconds
        (truncated if exceeded)
        "verbose": False,              # Print data preparation statistics
    },
    "device": "cuda",                 # "cuda" or "cpu"
    "verbose": False,                 # Print training statistics
    "plot_histograms": False,         # Plot conditional histograms
}
```

```

    "plot_pp": False,                # Plot P-P (probability-probability)
    plots
    "history_length": 32,            # History length k=1 in number of IEIs
}

```

By default, the same history length is used for both source and target: $k = l = \text{history_length}$. The HPC pipeline uses Optuna to automatically find the best hyperparameters for each neuron pair.

- `seed` (*optional*): Integer random seed for dataset partitioning and model initialization. The same seed produces near-identical results.

Outputs

- `(TE_test, ln_yy_test, ln_yyx_test)`:
 - `TE_test`: Estimated transfer entropy rate in **nats per second**.
 - `ln_yy_test`: Mean log-ratio from the reduced model on the test set.
 - `ln_yyx_test`: Mean log-ratio from the full model on the test set.
 - Relationship: `TE_test = ln_yyx_test - ln_yy_test`.
- `(log_loss_yy, log_loss_yyx)`: Negative log-likelihood loss of the reduced and full models on the test set, respectively. Lower values indicate better model fit.

If an exception occurs during training or estimation, the function returns `NaN` values along with an error message.

`Ln_estimation_yy` and `Ln_estimation_yyx`

These two functions handle sub-tasks within `TE_estimation_tpp`. They train the reduced model (`yy`) and the full model (`yyx`) respectively, and return the corresponding log-ratio and test loss.

- `Ln_estimation_yy(event_time, configs, seed, trial)`: Trains on the **target only**; returns `(ln_yy_test, log_loss_yy)`.
- `Ln_estimation_yyx(event_time, configs, seed, trial)`: Trains on **target + source**; returns `(ln_yyx_test, log_loss_yyx)`.

These functions are called directly by the HPC scripts (`hyper_opt_reduced.py`, `hyper_opt_full.py`, `multi_runs_reduced.py`, `multi_runs_full.py`).

Workflow Pipeline (Local)

This workflow runs the complete TE estimation locally for a single pair of processes. Refer to `demo/demo.ipynb` for a fully worked example.

Step 1 — Prepare the environment

```

# Install this project
pip install -e .
# Install dependent packages using conda:
conda env create -f environment.yml
conda activate RMDN-TE

```

Step 2 — Prepare input data

Provide event times as a list of `torch.Tensor` objects (in seconds). If your data is in MATLAB `.mat` format, convert it first.

Step 3 — Configure and run

```
import torch
from src.te_tpp import TE_estimation_tpp

event_time = [
    torch.tensor([2., 4., 6., ...]), # Target process
    torch.tensor([1., 3., 5., ...]), # Source process
]

configs = { ... } # See configs description above

(TE_test, ln_yy_test, ln_yyx_test), (log_loss_yy, log_loss_yyx) = \
    TE_estimation_tpp(event_time, configs, seed=42)

print(f"TE rate: {TE_test:.4f} nats/sec")
```

Step 4 — Evaluate

Assess the model fit using the returned log-loss values and optional P-P plots (set `"plot_pp": True` in `configs`). A well-fitted model should show points close to the diagonal in the P-P plot. Repeat with different seeds to assess estimation variance.

Workflow Pipeline (HPC)

The HPC pipeline estimates TE for all pairwise neuron combinations in a dataset using SLURM array jobs. There are two parallel tracks — **reduced** (target self-prediction) and **full** (source + target prediction) — each consisting of two sequential steps: **hyperparameter optimization**, then **multi-run estimation**.

```
Input HDF5
|
|→ [Step 0] Generate task lists (tasks_reduced.csv, tasks_full.csv)
|
|→ [Step 1a] Hyperparameter optimization – Reduced model
(opt_reduced_model_batch.sh)
|→ [Step 1b] Hyperparameter optimization – Full model
(opt_full_model_batch.sh)
|
|→ [Step 2a] Multi-run estimation – Reduced model
(run_reduced_model_batch.sh)
|→ [Step 2b] Multi-run estimation – Full model
(run_full_model_batch.sh)
|
|→ [Step 3] Aggregate results
(aggregate_results.py)
|→ [Step 4] Calculate TE
(calculate_te.py)
|→ [Step 5] Visualize
(plot_te_heatmap.py)
```

Data Conversion

If your spike data is in MATLAB format, convert it to HDF5 first. The script reads a `sortedData` cell array, filters neurons with fewer than `min_events` spikes, and organises the output as `group_id/neuron_id` datasets:

```
python hpc/mat_to_h5.py
# Converts data/testFile.mat → data/event_times_data.h5 (min_events=1500)
```

The HDF5 structure produced is:

```
event_times_data.h5
└─ <group_id>/          # e.g. "BC", "POm"
    └─ <neuron_id>/      # e.g. "0", "1", "2", ...
        └─ [float32 array of event times in seconds]
```

Step 0 — Generate Task Lists

Before submitting SLURM jobs, generate the task CSV files that map each SLURM array task ID to a specific neuron (or neuron pair):

```
python hpc/hpc_header.py
```

This is interactive:

- For the **reduced model**, you select which neuron groups to include; one task is created per neuron: `tasks_reduced.csv`.
- For the **full model**, you select source and target groups; one task is created per ordered (source, target) neuron pair across different groups: `tasks_full.csv`.

Step 1 — Hyperparameter Optimization

Run Optuna-based hyperparameter search using SLURM array jobs. Each task runs 50 trials to find the best model architecture and regularization settings for that specific neuron (or neuron pair). Results are saved as JSON-formatted `.txt` files in `results/opt-reduced/` and `results/opt-full/`.

```
# Reduced model (one task per target neuron)
sbatch hpc/opt_reduced_model_batch.sh

# Full model (one task per source-target neuron pair)
sbatch hpc/opt_full_model_batch.sh
```

Key SLURM parameters (edit in the `.sh` files to match your cluster and dataset):

Parameter	Reduced	Full	Description
<code>--array</code>	0-1	0-25	Task ID range (one per neuron/pair)
<code>--time</code>	01:00:00	01:00:00	Wall time per job
<code>--gres</code>	gpu:1	gpu:1	GPUs per job
<code>TASKS_PER_JOB</code>	16	16	Concurrent processes per GPU (via MPS)
<code>--history_length</code>	128	128	IEI history length $k = l$
<code>--num_trials</code>	20	20	Optuna trials per task

Each job launches `TASKS_PER_JOB` Python processes in parallel via CUDA Multi-Process Service (MPS), which allows multiple processes to share a single GPU efficiently.

Verify completion:

```
python hpc/check_results.py --reduced <N_reduced_tasks> --full <N_full_tasks>
```

Step 2 — Multi-Run Estimation

Using the best hyperparameters found in Step 1, run the estimation `num_runs` times per task to quantify estimation variance. Results are saved as CSV files in `results/runs-reduced/` and `results/runs-full/`. The script supports **resuming** interrupted runs automatically.

```
# Reduced model
sbatch hpc/run_reduced_model_batch.sh

# Full model
sbatch hpc/run_full_model_batch.sh
```

Key script arguments (edit in the `.sh` files):

Argument	Example	Description
<code>--num_runs</code>	20	Number of independent estimation runs per task
<code>--history_length</code>	128	Must match the value used in Step 1
<code>--data_time_length</code>	900	Total recording duration in seconds
<code>--seed</code>	42	Base random seed

Verify completion:

```
python hpc/check_results.py --reduced <N> --full <N> --num_runs 20
```

Step 3 — Aggregate Results

Consolidate all per-task CSV files from Step 2 into a single HDF5 file for efficient downstream processing:

```
python hpc/aggregate_results.py
# Output: results/multi_runs_results.h5
```

The output HDF5 file has the structure:

```
multi_runs_results.h5
├─ reduced/
│   └─ <group_id>_<neuron_id>/    # e.g. "BC_0"
│       ├── ln_yy_sec              # Log-ratio values per run
│       └─ run_duration_sec        # Runtime per run
└─ full/
    └─ <src_group>_<src_id>_to_<tgt_group>_<tgt_id>/    # e.g. "BC_0_to_P0m_3"
        ├── ln_yyx_sec
        └─ run_duration_sec
```

Step 4 — Calculate Transfer Entropy

Compute the final TE rate for each source-target pair using the aggregated results, along with uncertainty statistics:

```
python hpc/calculate_te.py
# Output: results/te_results_hpc.csv
```

The output CSV contains one row per source-target neuron pair with the following columns:

Column	Description
task_id	Task identifier
source_group_id, source_neuron_id	Source neuron identity
target_group_id, target_neuron_id	Target neuron identity
te_mean	Mean TE rate across runs (nats/sec)
te_std	Standard deviation across runs
te_sem	Standard error of the mean
te_iqr	Interquartile range
te_ci_lower, te_ci_upper	95% bootstrap confidence interval bounds
te_snr_linear	Signal-to-noise ratio (mean / std)
te_snr_db	SNR in decibels ($20 \times \log_{10}(\text{SNR})$)
mean_runtime	Average runtime per estimation run (seconds)

Step 5 — Visualize

Generate heatmap plots of the pairwise TE estimates:

```
python hpc/plot_te_heatmap.py
# Output: results/te_heatmap_standard.png, results/te_heatmap_95ci.png
```

Two heatmaps are produced side by side (e.g., BC→POM and POM→BC directions), with neurons on each axis and TE magnitude shown by color. A shared colorbar is used for direct comparison.

Metrics for Evaluation

Model Fit

- **P-P Plot (Probability-Probability Plot):** Compares the empirical CDF of observed IEIs against the model's predicted CDF. Points close to the diagonal indicate a well-calibrated model. Enable with `"plot_pp": True` in `configs`. Output files follow the pattern `pp_plot_<model_name>_<seed>.png`.
- **Negative Log-Likelihood Loss** (`log_loss_yy`, `log_loss_yyx`): Lower values indicate the model assigns higher probability to observed events. Used during hyperparameter optimization as the objective to minimize.

Estimation Reliability

- **Standard Deviation and Standard Error:** Computed across the `num_runs` independent estimation runs. High variance relative to the mean suggests insufficient data or model instability.
- **95% Bootstrap Confidence Interval** (`te_ci_lower`, `te_ci_upper`): Constructed via 2000 bootstrap resamples of the per-run TE estimates. A confidence interval that excludes zero provides evidence of a genuine information flow.
- **Interquartile Range (IQR):** A robust spread measure less sensitive to outlier runs.
- **SNR (dB):** Signal-to-noise ratio expressed in decibels. Higher values indicate more reliable detection of a non-zero TE. Computed as $20 \cdot \log_{10}(\mu/\sigma)$.
- **Box plots** (as shown in `results/local_runs/`): Visual summaries of the TE distribution across runs, useful for detecting systematic bias or heavy-tailed variance.

Requirements

Use `pip install -e .` to install the project package and all dependencies defined in `setup.py`.

For GPU support, install the CUDA-enabled PyTorch build:

```
conda env create -f environment.yml
conda activate RMDN-TE
```

Core dependencies:

Package	Version	Purpose
Python	3.12	Language runtime
PyTorch	2.5.1	Deep learning framework
pytorch-cuda	12.4	GPU acceleration (optional)
NumPy	2.2.5	Numerical computation
SciPy	1.15.3	Statistical utilities
scikit-learn	1.6.1	Data splitting utilities
Optuna	4.4.0	Hyperparameter optimization
pandas	2.2.3	Data manipulation and CSV I/O
h5py	3.15.1	HDF5 file I/O
matplotlib / seaborn	—	Plotting
tqdm	—	Progress bars
juliacall	0.9.28	Julia interop for CoTETE comparison (optional)

HPC cluster environment (bwUniCluster 2.0 / bwForCluster):

```
module load devel/miniforge/24.9.2
module load devel/cuda/12.6
```